Computer Science & information Technology

Compiler Design

DPP: 1

Syntax Directed Translation

Q1 Synthesized attributes can be easily simulated using
(A) LL grammar
(B) LR grammar
(C) Ambiguous grammar
(D) None of these

Q2 Consider the following translation rules for the grammar G:

S → a {print "A"} A
A → b {print "C"}B
A → ∈ {print "C"}
B → e {print "B"}A
B → ∈ {print "C"}
C → c {print "A"}

What will be the output for the input string "abebebe" you top-down parser?
(A) ACBCCBAC (B) ACCBCCBC
(C) ACBCCBCC (D) ACBCBCBC

Q3 Consider the following attribute grammar:

A → BA′ A′· b = A · a
A · a = A′ · b
$A_1' \rightarrow +BA_2'$ A_2' . b = A_1' . b + B · a
A_1' . a = A_2' . a

Which of the following is true?
(A) Both a and b are inherited attributed.
(B) Both a and b are synthesized attributed.
(C) a is inherited, b is synthesized
(D) b is inherited, a is synthesized

Q4 Consider the following grammar:

E → E + T | T
T → T/F | F
F → F * A | A
A → id

Which one of the following is/are true?
(A) / have higher precedence than *
(B) *have higher precedence than +
(C) + have lower precedence than/
(D) *, +, / all have same precedence.

Q5 A shift reduce parser perform action specified within process immediately after reduction to the corresponding rule of grammar.

S → abV {print '11'}
S → cc {print '22'}
V → Sd {print '33'}

What is the translation of "ababccdd" using the SDT scheme described by above rules?
(A) 2233113311 (B) 1133113322
(C) 2211331122 (D) 1122113322

Q6 Consider the following transition rules:

A → BC
C → +BC | A + | ∈
B → DB {print '+'} | ∈
D → id {print number value}

If input is given "2 + 34" then his translation scheme will generate output.
(A) 2 + 3 + 4 +
(B) + 2 + 3 + 4
(C) + + 2 + 34
(D) 2 + 34 ++

Q7 ____is performed by attaching rules or algorithms to production in a grammar.
(A) Lexical analysis

(B) Execution
(C) Syntax directed translation
(D) None of these.

Q8 Consider a translation scheme is given as:

$S \rightarrow S_1 + S_2$ $\{S \cdot val = S_1 \cdot val + S_2 \cdot val\}$

$S \rightarrow S_1 * S_2$ $\{S \cdot val = S_1 \cdot val * S_2 \cdot val\}$

$S \rightarrow id$ $\{S \cdot val = id\}$

What will be the output for 5 * 6 + 7?

(A) 18
(B) 37
(C) 65
(D)

Cannot be identified because it is ambiguous grammar.

Q9 Consider the following SDT :

$S \rightarrow E$ $\{S \cdot val = E \cdot val\}$

$E \rightarrow E + T$ $\{E \cdot val\} = E_1 \cdot val + T \cdot val\}$

$E \rightarrow T$ $\{E \cdot val = T \cdot val\}$

$T \rightarrow T \times F$ $\{T \cdot val = T_1 \cdot val * f \cdot val\}$

$T \rightarrow F$ $\{T \cdot val = f \cdot val\}$

$F \rightarrow (E)$ $\{F \cdot val = E \cdot val\}$

$F \rightarrow a$ $\{f \cdot val = a\}$

What will be the output of the expression "20 + 8 × 6"

Answer Key

Q1	(B)
Q2	(D)
Q3	(D)
Q4	(B, C)
Q5	(A)
Q6	(D)
Q7	(C)
Q8	(D)
Q9	68

Android App | iOS App | PW Website

Hints & Solutions

Q1 Text Solution:

Synthesized attributes can be easily simulated by LR grammar.

As Synthesized attribute are evaluated in bottom-up manner and LR grammar is also evaluated in bottom-up manner.

Q2 Text Solution:

After evaluating oputput will be ACBCBCBC.

Therefore, option (d) correct.

Q3 Text Solution:

For the first production a, b are attributes, such that

$a \in A$, and $b \in A'$.

Now if you look into the translation closely,

$A' \cdot b = A \cdot a$ (b attribute is taking value from its parent attribute)

$A' \cdot a = A' \cdot b$ (a attribute is taking value from one of its child attributes)

Similarly for second production, b is taking values from its parent as will as sibling which left to it (it is L -attributed as well as S)

So, (b) follows inherited attribute definition while (a) follows synthesized attribute definition.

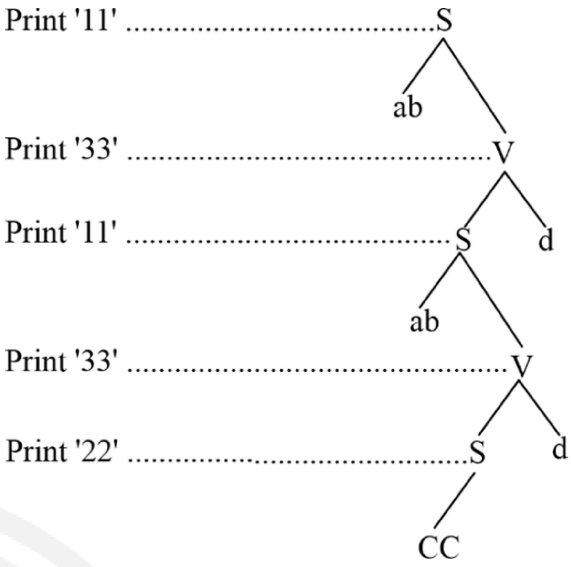
Q4 Text Solution:

*have higher precedence than +

+ have lower precedence than/

Q5 Text Solution:

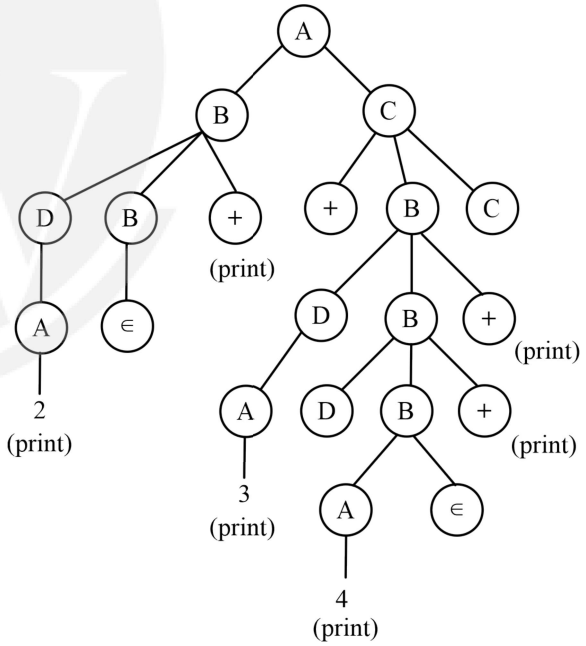
Given input string ababccdd



So, output will be 2233113311. Therefore option (a) is correct.

Q6 Text Solution:

The input given is "2 + 34"



On scanning them left to right.

Output = 2 + 34 ++

So, option (d) is correct.

Q7 Text Solution:

SDT is done by attaching augmented rules to the analysis. SDT rules used the following:

1. Lexical value of node
2. Constants
3. Attributes associated to the non-terminal in their definition.

Q8 **Text Solution:**

The given grammar is ambiguous grammar

S → S1 + S2 | S1 * S2 | id an ambiguous grammar.

Because for expression 5 * 6 + 7 two output are possible.

Output 1 : 37

Output 2 : 65

So option (d) is correct answer.

Q9 **Text Solution:**

Output will be 20 + 6*8 = 68


Android App | iOS App | PW Website